OpenRun

Self-hosted GitOps for *web apps* and *internal tools*

Deploy to Docker, Podman or Kubernetes from one declarative config in Git. Single sign-on, access control, audit logs, TLS and scale-to-zero are built in.

open source | Apache-2.0 | single binary | docker | podman | kubernetes

bash

```
# install and start the server
$ curl -sSL https://openrun.dev/install.sh | sh
$ openrun server start &

# deploy every app declared in a Git repo
$ openrun sync schedule --approve --promote \
    github.com/openrundev/openrun/examples/utils.star
$
```

openrun.dev github.com/openrundev/openrun

introduction

What is OpenRun?

A platform for deploying web apps on your own infrastructure. Teams use it to run internal tools with **login, access control and auditing** on every app.

RUNS ON

docker · podman · kubernetes

MANAGED THROUGH

git · cli · web console · rest api · mcp

OPEN SOURCE

Self-hosted

An alternative to Google Cloud Run and Heroku that runs on your own servers, inside your network.

GITOPS

Declarative

Apps are created, configured and promoted by editing a config file in Git. OpenRun reconciles the rest.

OPERATIONS

One binary

Web server, TLS, routing, auth and deployment in a single Go binary. No separate proxy or CI/CD system.

SCOPE

Focused

Built for web apps and internal tools. A small surface keeps the platform simple to run and to learn.

platform overview

Key features

GITOPS

Declarative deploys

Create, configure and promote apps by editing one file in Git. No UI clicks, no deploy scripts.

AUTH

SSO and RBAC

OAuth, OIDC, SAML or client certificates in front of any app, with role-based access. No app code changes.

BINDINGS

Service bindings

Each app gets its own database account. PostgreSQL, MySQL, SQLite and Redis are built in, more through providers.

SCALE

Scale to zero

Idle app containers stop and start again on the next request. Run many apps on small hardware.

RELEASES

Staged releases

Changes land in a staging environment first, then promote to production with a blue-green switch.

STORAGE

Durable SQLite

SQLite volumes replicate continuously with Litestream to S3-compatible storage and restore after node loss.

ACTIONS

Actions and MCP

Generated forms for backend operations. Actions also run from the CLI and are served as MCP tools.

AUDIT

Audit and telemetry

Every operation and API call is recorded as an audit event. Traces and metrics export over OpenTelemetry.

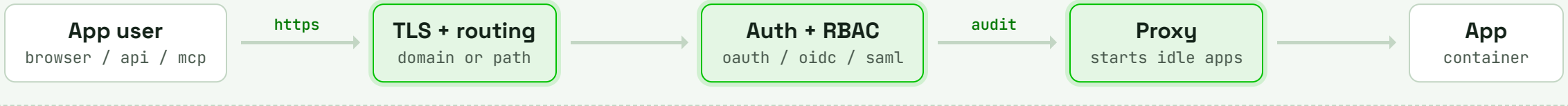
system design

How it works

DEPLOY: FROM GIT TO A RUNNING APP



SERVE: EVERY REQUEST PASSES THROUGH THE SERVER



One Go binary

The same process reconciles state from Git and routes incoming requests.

Small footprint

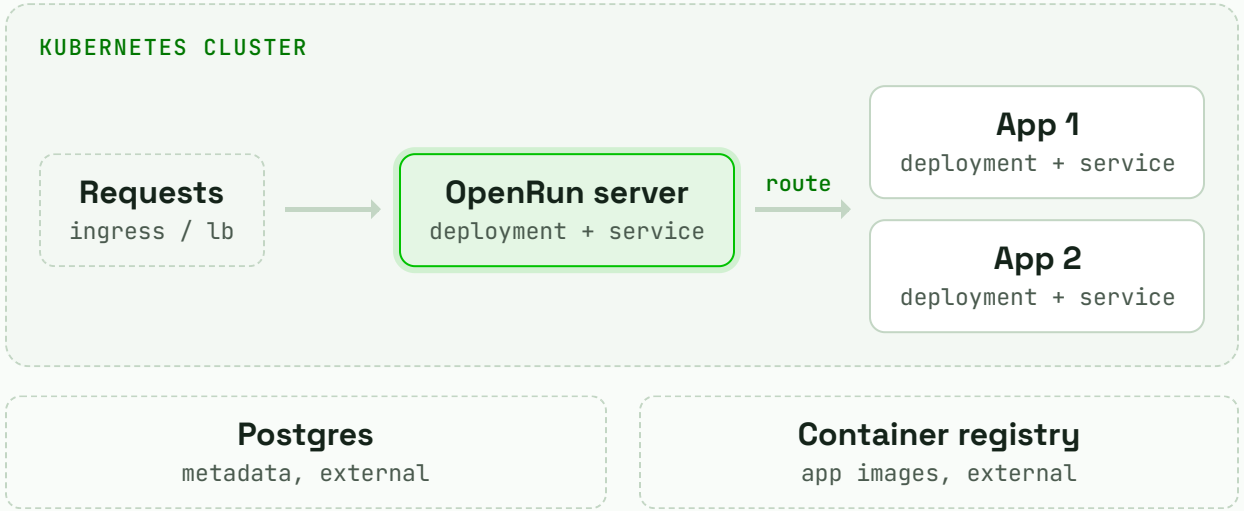
A single node uses an embedded SQLite database for metadata. Kubernetes uses Postgres.

Security at the platform

Login, access rules and audit events are handled before a request reaches the app.

kubernetes deployment

OpenRun on Kubernetes



COMPONENTS

A standard Helm install

OpenRun runs as a Deployment with a Service. It uses Postgres for metadata and a registry for the images it builds.

ROUTING

No per-app Ingress

Requests reach the OpenRun server, which routes to each app through its ClusterIP Service.

RECONCILE

Lazy creation, server-side apply

App resources are created on the first request and updated with Kubernetes server-side apply.

getting started

Three ways to install

A / SINGLE NODE

Linux, macOS or Windows with Docker or Podman. The quickest way to start.

```
$ curl -sSL https://openrun.dev/install.sh | sh   # or: brew install openrun
$ openrun server start &
```

B / KUBERNETES

A Helm chart for an existing cluster. Apps scale across nodes with the same config.

```
$ helm repo add openrun https://openrundev.github.io/openrun-helm-charts/
$ helm install openrun1 openrun/openrun \
    --namespace openrun --create-namespace --set registry.enabled=true
```

C / TERRAFORM ON AWS

Creates an EKS cluster, an RDS database and an ECR registry, then installs OpenRun.

```
$ cd openrun/deploy/terraform
$ terraform init && terraform apply --var-file terraform.tfvars
```

NOTE For production on Kubernetes, use an external Postgres database and container registry.

deployment workflow

GitOps in one command

```bash
$ openrun sync schedule --approve --promote \
    github.com/openrundev/openrun/examples/utils.star
```

`--approve` accept the permissions each app asks for

`--promote` move staged changes to production

After this, a commit to the repo is the only deploy step.

WHAT THE SCHEDULED SYNC DOES

01 WATCH

Poll the repo

Checks the Git URL for new commits to config or source.

02 READ

Load config

Reads the app declarations and fetches each app's source.

03 BUILD

Build images

Builds from an AppSpec or the app's own Dockerfile.

04 DEPLOY

Apply changes

Creates new apps and updates existing ones, staging first.

05 SECURE

Enforce policy

Applies the configured login, access rules and audit logging.

simplicity first

The whole config, side by side

apps.star — openrun / starlark

```
# Resource limits (optional)
limits = {"cpus": "1", "memory": "512m"}

# A Streamlit app, served at /misc/streamlit_example
app(path="/misc/streamlit_example",
    source="github.com/streamlit/streamlit-example",
    git_branch="master", spec="python-streamlit",
    container_opts=limits)

# A FastHTML app, served at the fasthtml subdomain
app(path="fasthtml.:",
    source="github.com/AnswerDotAI/fasthtml/examples",
    spec="python-fasthtml",
    params={"APP_MODULE": "basic_ws:app"},
    container_opts=limits)
```

Complete for two apps: where the source is, how to build it, how to route to it and what limits to set.

app.yaml — kubernetes / yaml, 94 lines, scroll

```
# Deployment: runs the app container
apiVersion: apps/v1
kind: Deployment
metadata:
  name: streamlit-app
  namespace: apps
  labels:
    app: streamlit
spec:
  replicas: 1
  selector:
    matchLabels:
      app: streamlit
  template:
    metadata:
      labels:
        app: streamlit
    spec:
      containers:
        - name: streamlit
          image: registry.example.com/apps/streamlit-example:1.0.0
          ports:
            - containerPort: 8501
          resources:
            limits:
              cpu: "1"
              memory: 512Mi
          readinessProbe:
            httpGet:
```

One app, without RBAC or auditing. Argo CD, an IDP, SSO, cert-manager and a CI image build also need to be configured.

comparison

OpenRun and managed cloud services

COMPARED WITH Google Cloud Run AWS App Runner

WHERE OPENRUN IS STRONGER

+ **Your network, your data**
  Runs on your own hardware or VPC, so network policy and data location stay under your control.

+ **Predictable cost**
  No per-request billing. Busy apps and custom hardware run on capacity you already pay for.

+ **Login without IAM plumbing**
  Add SSO and role-based access to any app through config, with no sidecars or app changes.

+ **Everything in Git**
  The deployment setup is versioned and reviewable, not spread across cloud console settings.

TRADE-OFFS

! **You run the infrastructure**
  Hardware, OS updates and uptime are yours. There is no provider SLA underneath.

! **No global edge**
  Multi-region failover and a worldwide CDN need to be set up separately.

! **Fewer managed services**
  Service bindings cover databases. Queues and other cloud services are integrated by you.

comparison

OpenRun and a DIY Kubernetes stack

COMPARED WITH `Argo CD / Flux` `cert-manager` `identity proxy` `CI pipelines` `Helm charts`

WHERE OPENRUN IS STRONGER

+ **One platform layer**
  GitOps, image builds, TLS and SSO come in one tool, in place of several you integrate yourself.

+ **Simple config for developers**
  A few lines of declarative config per app, with no Kubernetes YAML or Helm charts to write.

+ **Start small, grow later**
  Run on one node with Docker or Podman, then move to Kubernetes with the same config.

+ **Ready on day one**
  Auth, scale-to-zero and staged releases work without building an internal developer platform.

TRADE-OFFS

! **Advanced networking**
  A service mesh or multi-cluster networking calls for a custom Kubernetes setup.

! **Custom operators**
  Apps that depend on operators and CRDs for their logic are a better fit for raw Kubernetes.

! **Fine-grained scheduling**
  Node affinity, taints and anti-affinity rules are not exposed in the simplified config.

use cases

For teams

APPS

Internal web apps

Data science and tools teams deploy framework apps with little or no config. No infrastructure setup per app.

`Streamlit` `Gradio` `FastHTML` `NiceGUI` `FastAPI`

OPS

Operations automation

Replace Jenkins or Rundeck jobs with OpenRun Actions: scripts, maintenance tasks and runbooks behind a generated form.

`runbooks` `maintenance` `provisioning`

APIS

Secure internal APIs

Put login and audit logging in front of internal REST APIs and MCP servers without changing their code.

`OAuth` `OIDC / SAML` `audit events`

PROCESS

One way to deploy

Every team deploys through the same Git repo, so each change has history and goes through review.

`pull requests` `version history` `staging`

use cases

For individual developers

DEV

Local development

Run the same config on a laptop that runs in production. No Docker Compose files or manual networking.

`localhost` `dev/prod parity`

SHARE

Share apps safely

Host apps for friends and family. Add a login to a hobby or AI-generated app without writing auth code.

`OAuth login` `side projects`

HOMELAB

Efficient homelab

Idle apps stop until the next request, which suits a Raspberry Pi or older hardware running many services.

`scale to zero` `Raspberry Pi`

LEARN

Learn GitOps

Work with declarative deploys and reconciliation hands-on, without learning Kubernetes first.

`declarative` `reconciliation`

summary

Get started

OPENRUN IS A GOOD FIT WHEN

+ **You want cloud simplicity on your own hardware**
  Containers that build, route and scale to zero, on a server or VPS you control.

+ **You prefer config in Git**
  Apps and their settings are declared in a repo and changed through commits.

+ **Your apps need login and access control**
  SSO, RBAC and audit logs for every internal tool, without per-app work.

bash

```
# 1. install and start
$ curl -sSL https://openrun.dev/install.sh | sh
$ openrun server start &
# 2. sync apps from a Git repo
$ openrun sync schedule --approve --promote \
    github.com/openrundev/openrun/examples/utils.star
# 3. see what is running
$ openrun app list
```

Quick start | Star on GitHub | Console tour